GraphQL Case Study-31

Problem Statement

Build a GraphQL API to manage electricity consumers, meter readings, billing cycles, and payments.

Tech Stack / Frameworks & Libraries:

Layer	Technology
Language	Python 3.13
Web Framework	Starlette 0.27
GraphQL Library	Graphene 3.3
ORM (Database)	SQLAlchemy 2.0
Database	SQLite (file: **electricity_billing.db**)
Server	Uvicorn 0.23
GraphQL UI	GraphiQL (browser at `http://localhost:8000/graphql`)
Rate Cards	Residential: ₹7.5/unit · Commercial: ₹12/unit

Database Models:

Model	Fields
Consumer	id, name, address, connection_type (RESIDENTIAL/COMMERCIAL), status
Meter	id, consumer_id, meter_number, status
MeterReading	id, meter_id, reading_value, reading_date
Bill	id, consumer_id, billing_cycle, total_units, amount, status (GENERATED/PAID/OVERDUE)
Payment	id, bill_id, amount, payment_date, status

Mutations (Write Operations):

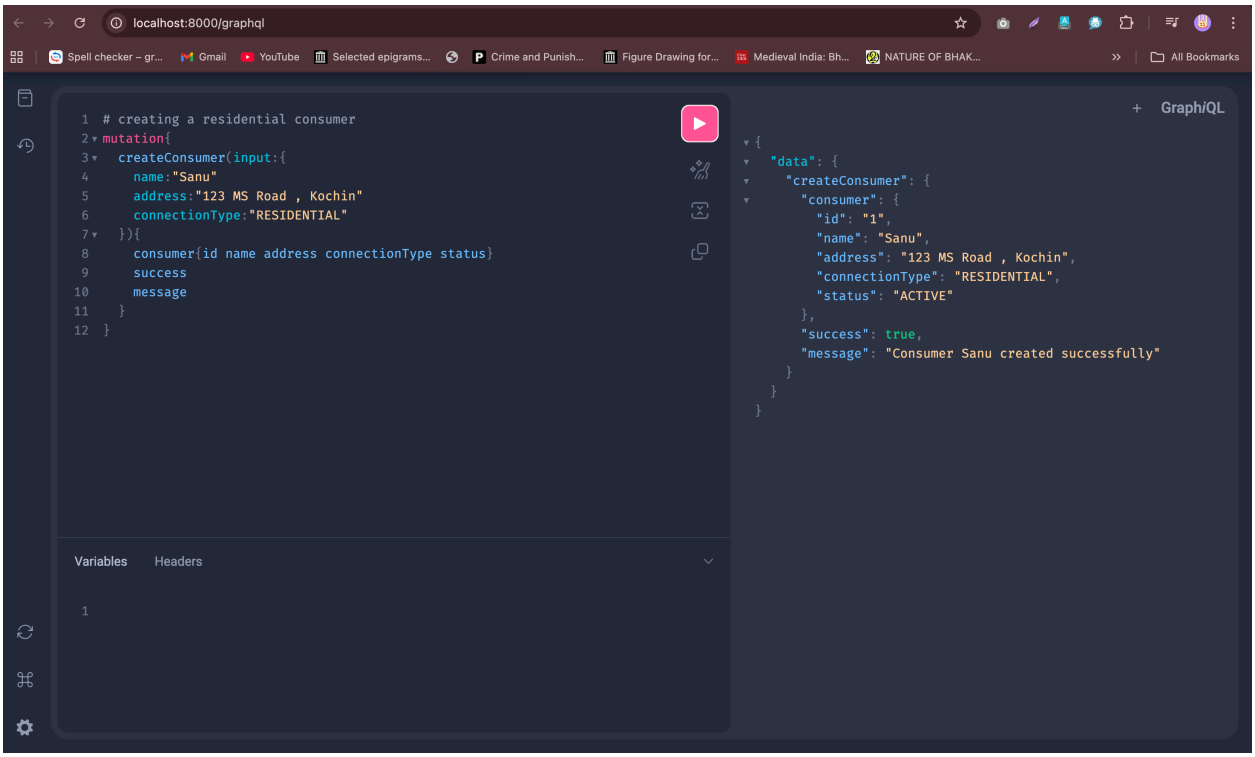
Mutation	What it does
`createConsumer`	Register a new consumer
`updateConsumer`	Update consumer details
`createMeter` / `deleteMeter`	Install/remove meters
`recordMeterReading`	Record electricity usage
`generateBill`	Auto-calculate and generate bill
`recordPayment`	Pay a bill (partial or full)
`checkOverdueBills`	Scan and mark all past-due bills as OVERDUE

Queries (Read Operations)

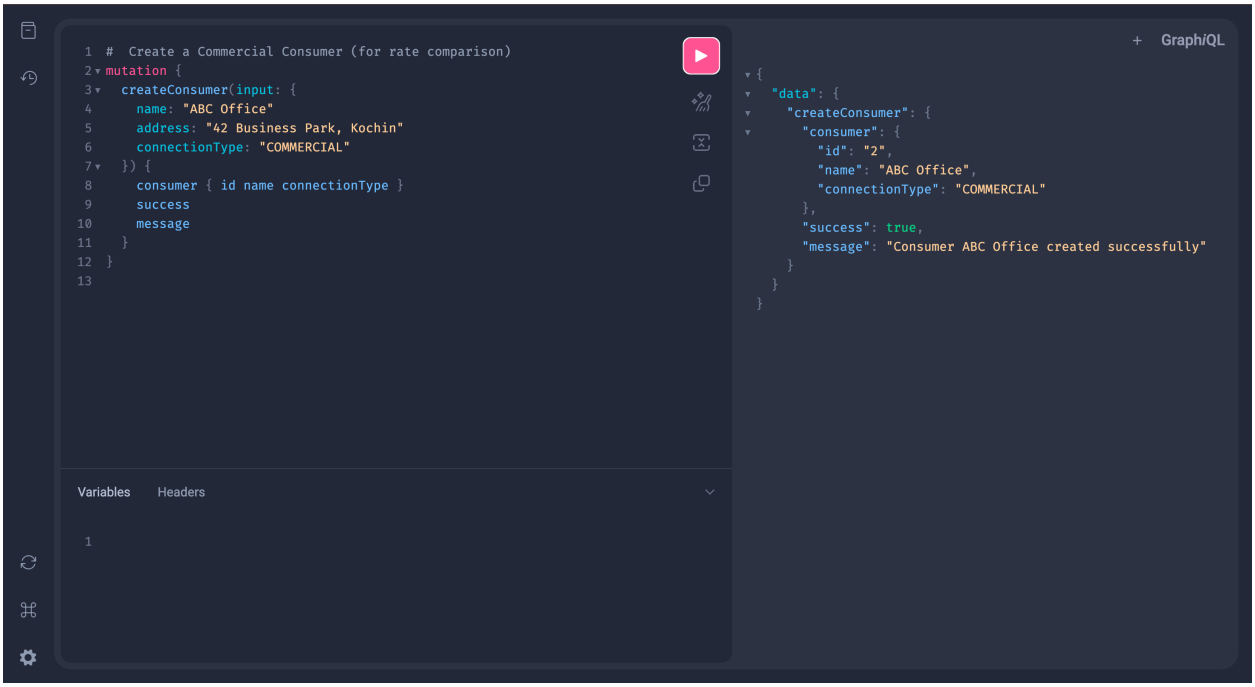
`allConsumers`, `allMeters`, `allMeterReadings`, `allBills`, `allPayments`, `billsByConsumer`, `billsByStatus`, `paymentsByBill`, `metersByConsumer`, `re`

#	Test Case	How
1	Unit calculation accuracy	`latest reading − previous reading = units consumed`
2	Bill generation validation	Blocks duplicate bills for same consumer + same cycle
3	Payment workflow	Tracks partial payments, marks bill PAID when fully settled
4	Overdue handling	`checkOverdueBills` scans all GENERATED bills past due date
5	Duplicate reading prevention	Blocks two readings for same meter on same day
6	Billing history maintained	`billsByConsumer` + `paymentsByBill` preserve full history

1 - Create a residential user



2- create a commercial Consumer



3 - **Create a Meter for Consumer 1**

```graphql
mutation {
  createMeter(input: {
    consumerId: 1
    meterNumber: "MTR-RES-001"
  }) {
    meter {
      id
      meterNumber
      consumerId
      status
      consumer {
        name
        connectionType
      }
    }
    success
    message
  }
}
```

Variables Headers

```
1
```

GraphiQL

```
{
  "data": {
    "createMeter": {
      "meter": {
        "id": "1",
        "meterNumber": "MTR-RES-001",
        "consumerId": 1,
        "status": "ACTIVE",
        "consumer": {
          "name": "Sanu",
          "connectionType": "RESIDENTIAL"
        }
      },
      "success": true,
      "message": "Meter created successfully"
    }
  }
}
```

Test cases-:

1) **Duplicate Reading Prevention**:

record First reading(baseline)

```graphql
mutation {
  recordMeterReading(input: {
    meterId: 1
    readingValue: 3000
  }) {
    meterReading { id readingValue readingDate }
    success
    message
    unitsConsumed
  }
}
```

Variables Headers

```
1
```

GraphiQL

```
{
  "data": {
    "recordMeterReading": {
      "meterReading": {
        "id": "1",
        "readingValue": 3000,
        "readingDate": "2026-03-15T09:17:12.460589"
      },
      "success": true,
      "message": "Reading recorded. Units consumed: 3000.0",
      "unitsConsumed": 3000
    }
  }
}
```

tried recording a 2nd reading on the same meter:

```
mutation {
  recordMeterReading(input: {
    meterId: 1
    readingValue: 3200
  }) {
    success
    message
    unitsConsumed
  }
}
```

Variables Headers

```
1
```

GraphiQL

```
{
  "data": {
    "recordMeterReading": {
      "success": false,
      "message": "Reading already recorded today. Cannot duplicate.",
      "unitsConsumed": 0
    }
  }
}
```

2)Unit Calculation Accuracy:

```
mutation {
  createConsumer(input: {
    name: "Saniya Kapure"
    address: "123 MG Road, Pune"
    connectionType: "RESIDENTIAL"
  }) {
    consumer { id name address connectionType status }
    success
    message
  }
}
```

Variables Headers

```
1
```

GraphiQL

```
{
  "data": {
    "createConsumer": {
      "consumer": {
        "id": "1",
        "name": "Saniya Kapure",
        "address": "123 MG Road, Pune",
        "connectionType": "RESIDENTIAL",
        "status": "ACTIVE"
      },
      "success": true,
      "message": "Consumer Saniya Kapure created successfully"
    }
  }
}
```

```
mutation {
  createConsumer(input: {
    name: "DEF Office"
    address: "42 Business Park, Pune"
    connectionType: "COMMERCIAL"
  }) {
    consumer { id name connectionType }
    success
    message
  }
}
```

Variables Headers

```
1
```

GraphiQL

```
{
  "data": {
    "createConsumer": {
      "consumer": {
        "id": "2",
        "name": "DEF Office",
        "connectionType": "COMMERCIAL"
      },
      "success": true,
      "message": "Consumer DEF Office created successfully"
    }
  }
}
```

```
mutation {
  createMeter(input: {
    consumerId: 1
    meterNumber: "MTR-RES-001"
  }) {
    meter { id meterNumber consumerId status }
    success
    message
  }
}
```

Variables Headers

```
1
```

GraphiQL

```
{
  "data": {
    "createMeter": {
      "meter": {
        "id": "1",
        "meterNumber": "MTR-RES-001",
        "consumerId": 1,
        "status": "ACTIVE"
      },
      "success": true,
      "message": "Meter created successfully"
    }
  }
}
```

```
mutation {
  recordMeterReading(input: {
    meterId: 1
    readingValue: 3000
  }) {
    success message unitsConsumed
  }
}
```

GraphiQL

```
{
  "data": {
    "recordMeterReading": {
      "success": true,
      "message": "Reading recorded. Units consumed: 3000.0",
      "unitsConsumed": 3000
    }
  }
}
```

due to duplicate prevention it prevents 2readings on the same day thus i have shifted the first reading on yesterday in the DB so that i can record the 2nd reading in the DB

command :

```
python3 -c "
import sqlite3, datetime
conn = sqlite3.connect('electricity_billing.db')
yesterday = (datetime.datetime.utcnow() - datetime.timedelta(days=1)).isoformat()
conn.execute('UPDATE meter_readings SET reading_date=? WHERE meter_id=1', (yesterday,))
conn.commit()
print('First reading moved to yesterday!')
conn.close()
"
```

```
(venv) saniyakapure@Saniyas-MacBook-Air-2 graphql-exm % python3 -c "
import sqlite3, datetime
conn = sqlite3.connect('electricity_billing.db')
yesterday = (datetime.datetime.utcnow() - datetime.timedelta(days=1)).isoformat()
conn.execute('UPDATE meter_readings SET reading_date=? WHERE meter_id=1', (yesterday,))
conn.commit()
print('First reading moved to yesterday!')
conn.close()
"
<string>:4: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to repres
ent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  yesterday = (datetime.datetime.utcnow() - datetime.timedelta(days=1)).isoformat()
First reading moved to yesterday!
(venv) saniyakapure@Saniyas-MacBook-Air-2 graphql-exm %
```

now we will record the 2nd reading :

```
mutation {
  recordMeterReading(input: {
    meterId: 1
    readingValue: 3500
  }) {
    meterReading { id readingValue }
    success
    message
    unitsConsumed
  }
}
```

Execute query (Ctrl-Enter)

```
{
  "recordMeterReading": {
    "meterReading": {
      "id": "2",
      "readingValue": 3500
    },
    "success": true,
    "message": "Reading recorded. Units consumed: 500.0",
    "unitsConsumed": 500
  }
}
}
```

Variables Headers

`unitsConsumed: 500` (3500 − 3000 = 500 units)

3)Bill Generation Validation:

```graphql
mutation {
  generateBill(input: {
    consumerId: 1
    billingCycle: "2025-03"
  }) {
    bill {
      id
      billingCycle
      totalUnits
      amount
      status
      generatedDate
      dueDate
      consumer {
        name
        address
        connectionType
      }
    }
    success
    message
  }
}
```

Variables Headers

GraphiQL

```json
{
  "data": {
    "generateBill": {
      "bill": {
        "id": "1",
        "billingCycle": "2025-03",
        "totalUnits": 500,
        "amount": 3750,
        "status": "GENERATED",
        "generatedDate": "2026-03-15T09:39:41.131012",
        "dueDate": "2026-03-30T09:39:41.131019",
        "consumer": {
          "name": "Saniya Kapure",
          "address": "123 MG Road, Pune",
          "connectionType": "RESIDENTIAL"
        }
      },
      "success": true,
      "message": "Bill generated: 500.0 units × ₹7.5 = ₹3750.0"
    }
  }
}
```

- `totalUnits: 500`
- `amount: 3750` (500 × ₹7.5 residential rate)
- `status: GENERATED`

Duplicate Bill Prevention → should FAIL

```graphql
mutation {
  generateBill(input: {
    consumerId: 1
    billingCycle: "2025-03"
  }) {
    success
    message
  }
}
```

GraphiQL

```json
{
  "data": {
    "generateBill": {
      "success": false,
      "message": "Bill already generated for cycle 2025-03"
    }
  }
}
```

4)Payment Workflow:

```graphql
mutation {
  recordPayment(input: {
    billId: 1
    amount: 3750
  }) {
    payment {
      id
      amount
      paymentDate
      status
      bill {
        billingCycle
        totalUnits
        amount
        consumer {
          name
          address
          connectionType
        }
      }
    }
    success
    message
    remainingAmount
  }
}
```

Variables Headers

GraphiQL

```json
{
  "data": {
    "recordPayment": {
      "payment": {
        "id": "1",
        "amount": 3750,
        "paymentDate": "2026-03-15T09:43:46.972219",
        "status": "SUCCESS",
        "bill": {
          "billingCycle": "2025-03",
          "totalUnits": 500,
          "amount": 3750,
          "consumer": {
            "name": "Saniya Kapure",
            "address": "123 MG Road, Pune",
            "connectionType": "RESIDENTIAL"
          }
        }
      },
      "success": true,
      "message": "Payment recorded. Bill fully paid!",
      "remainingAmount": 0
    }
  }
}
```

5)Overdue Handling:

```graphql
mutation {
  generateBill(input: {
    consumerId: 2
    billingCycle: "2025-01"
  }) {
    bill {
      id
      amount
      status
      dueDate
      consumer {
        name
        address
        connectionType
      }
    }
    success
    message
  }
}
```

GraphiQL

```json
{
  "data": {
    "generateBill": {
      "bill": {
        "id": "2",
        "amount": 12000,
        "status": "GENERATED",
        "dueDate": "2026-03-30T09:48:24.755574",
        "consumer": {
          "name": "DEF Office",
          "address": "42 Business Park, Pune",
          "connectionType": "COMMERCIAL"
        }
      },
      "success": true,
      "message": "Bill generated: 1000.0 units × ₹12.0 = ₹12000.0"
    }
  }
}
```

```graphql
mutation {
  checkOverdueBills {
    updatedCount
    success
    message
    overdueBills {
      id billingCycle amount status
      consumer { name }
    }
  }
}
```

```
Variables    Headers
```

```
GraphiQL
```

```
{
  "data": {
    "checkOverdueBills": {
      "updatedCount": 1,
      "success": true,
      "message": "1 bill(s) marked as OVERDUE",
      "overdueBills": [
        {
          "id": "2",
          "billingCycle": "2025-01",
          "amount": 12000,
          "status": "OVERDUE",
          "consumer": {
            "name": "DEF Office"
          }
        }
      ]
    }
  }
}
```

overdue list:

```graphql
query {
  billsByStatus(status: "OVERDUE") {
    id consumerId billingCycle amount dueDate status
    consumer { name }
  }
}
```

```
GraphiQL
```

```
{
  "data": {
    "billsByStatus": [
      {
        "id": "2",
        "consumerId": 2,
        "billingCycle": "2025-01",
        "amount": 12000,
        "dueDate": "2026-02-23T09:51:18.310984",
        "status": "OVERDUE",
        "consumer": {
          "name": "DEF Office"
        }
      }
    ]
  }
}
```

6)Billing History Maintained:

```
query {
  billsByConsumer(consumerId: 1) {
    id
    billingCycle
    totalUnits
    amount
    status
    generatedDate
    dueDate
    consumer {
      name
      address
      connectionType
    }
  }
}
```

Variables Headers

GraphiQL


```
{
  "data": {
    "billsByConsumer": [
      {
        "id": "1",
        "billingCycle": "2025-03",
        "totalUnits": 500,
        "amount": 3750,
        "status": "PAID",
        "generatedDate": "2026-03-15T09:39:41.131012",
        "dueDate": "2026-03-30T09:39:41.131019",
        "consumer": {
          "name": "Saniya Kapure",
          "address": "123 MG Road, Pune",
          "connectionType": "RESIDENTIAL"
        }
      }
    ]
  }
}
```


```
query {
  paymentsByBill(billId: 1) {
    id
    amount
    paymentDate
    status
  }
}
```

GraphiQL


```
{
  "data": {
    "paymentsByBill": [
      {
        "id": "1",
        "amount": 3750,
        "paymentDate": "2026-03-15T09:43:46.972219",
        "status": "SUCCESS"
      }
    ]
  }
}
```


to check the database :

```
python3 -c "
import sqlite3
conn = sqlite3.connect('electricity_billing.db')
conn.row_factory = sqlite3.Row
print('=== CONSUMERS ===')
for r in conn.execute('SELECT * FROM consumers'): print(dict(r))
print('\n=== METERS ===')
for r in conn.execute('SELECT * FROM meters'): print(dict(r))
print('\n=== METER READINGS ===')
for r in conn.execute('SELECT * FROM meter_readings'): print(dict(r))
print('\n=== BILLS ===')
for r in conn.execute('SELECT * FROM bills'): print(dict(r))
print('\n=== PAYMENTS ===')
```

```
for r in conn.execute('SELECT * FROM payments'): print(dict(r))
conn.close()
"
```

```
Problems    Output    Debug Console    Terminal    Ports

saniyakapure@Saniyas-MacBook-Air-2 graphql-exm % python3 -c "
import sqlite3
conn = sqlite3.connect('electricity_billing.db')
conn.row_factory = sqlite3.Row
print('=== CONSUMERS ===')
for r in conn.execute('SELECT * FROM consumers'): print(dict(r))
print('\n=== METERS ===')
for r in conn.execute('SELECT * FROM meters'): print(dict(r))
print('\n=== METER READINGS ===')
for r in conn.execute('SELECT * FROM meter_readings'): print(dict(r))
print('\n=== BILLS ===')
for r in conn.execute('SELECT * FROM bills'): print(dict(r))
print('\n=== PAYMENTS ===')
for r in conn.execute('SELECT * FROM payments'): print(dict(r))
conn.close()
"
=== CONSUMERS ===
{'id': 1, 'name': 'Saniya Kapure', 'address': '123 MG Road, Pune', 'connection_type': 'RESIDENTIAL', 'status': 'ACTIVE'}
{'id': 2, 'name': 'DEF Office', 'address': '42 Business Park, Pune', 'connection_type': 'COMMERCIAL', 'status': 'ACTIVE'}

=== METERS ===
{'id': 1, 'meter_number': 'MTR-RES-001', 'status': 'ACTIVE', 'consumer_id': 1}
{'id': 2, 'meter_number': 'MTR-COM-001', 'status': 'ACTIVE', 'consumer_id': 2}

=== METER READINGS ===
{'id': 1, 'reading_value': 3000.0, 'reading_date': '2026-03-14T09:31:21.129605', 'meter_id': 1}
{'id': 2, 'reading_value': 3500.0, 'reading_date': '2026-03-15 09:35:24.892091', 'meter_id': 1}
{'id': 3, 'reading_value': 1000.0, 'reading_date': '2026-03-15 09:48:04.878131', 'meter_id': 2}

=== BILLS ===
{'id': 1, 'billing_cycle': '2025-03', 'total_units': 500.0, 'amount': 3750.0, 'status': 'PAID', 'generated_date': '2026-0
3-15 09:39:41.131012', 'due_date': '2026-03-30 09:39:41.131019', 'consumer_id': 1}
{'id': 2, 'billing_cycle': '2025-01', 'total_units': 1000.0, 'amount': 12000.0, 'status': 'OVERDUE', 'generated_date': '2
026-03-15 09:48:24.755563', 'due_date': '2026-02-23T09:51:18.310984', 'consumer_id': 2}

=== PAYMENTS ===
{'id': 1, 'amount': 3750.0, 'payment_date': '2026-03-15 09:43:46.972219', 'status': 'SUCCESS', 'bill_id': 1}
saniyakapure@Saniyas-MacBook-Air-2 graphql-exm %
```

Conclusion - :

- Used **Starlette** over FastAPI since it's lighter and perfectly suited for a pure GraphQL API
- Used **SQLite** for simplicity (no setup needed, file-based database)
- Used **GraphQL UI** for live testing